

多媒体第三次实验

1. 完成本次实验任务的思路

本次实验涵盖了对BMP位图文件的底层解析与位深转换，要求全程使用面向对象的方式（`class`）实现，不能使用如opencv、PIL等Python图像处理相关的模块或包。具体思路如下：

- **BMP图像的解析与保存思路：**

- 定义 `BMPImage` 类，封装图像的宽高、位深、调色板和像素数据。
- 使用内置的 `open(..., 'rb')` 接口读取二进制文件，并利用 `int.from_bytes` 解析文件头（14字节）和信息头（40字节）以获取图像的核心元数据。
- 根据位深（`bit_count`）判断是否需要读取由256个表项（每项4字节）组成的调色板。
- 定位至像素偏移处，严格按照每行字节数必须是4的倍数的公式 `(width * bit_count + 31) // 32 * 4` 读取数据，提取通道或索引。写入文件时执行逆操作，补齐对齐用的空字节以防图像扭曲。

- **24位真彩色转8位灰度任务：**

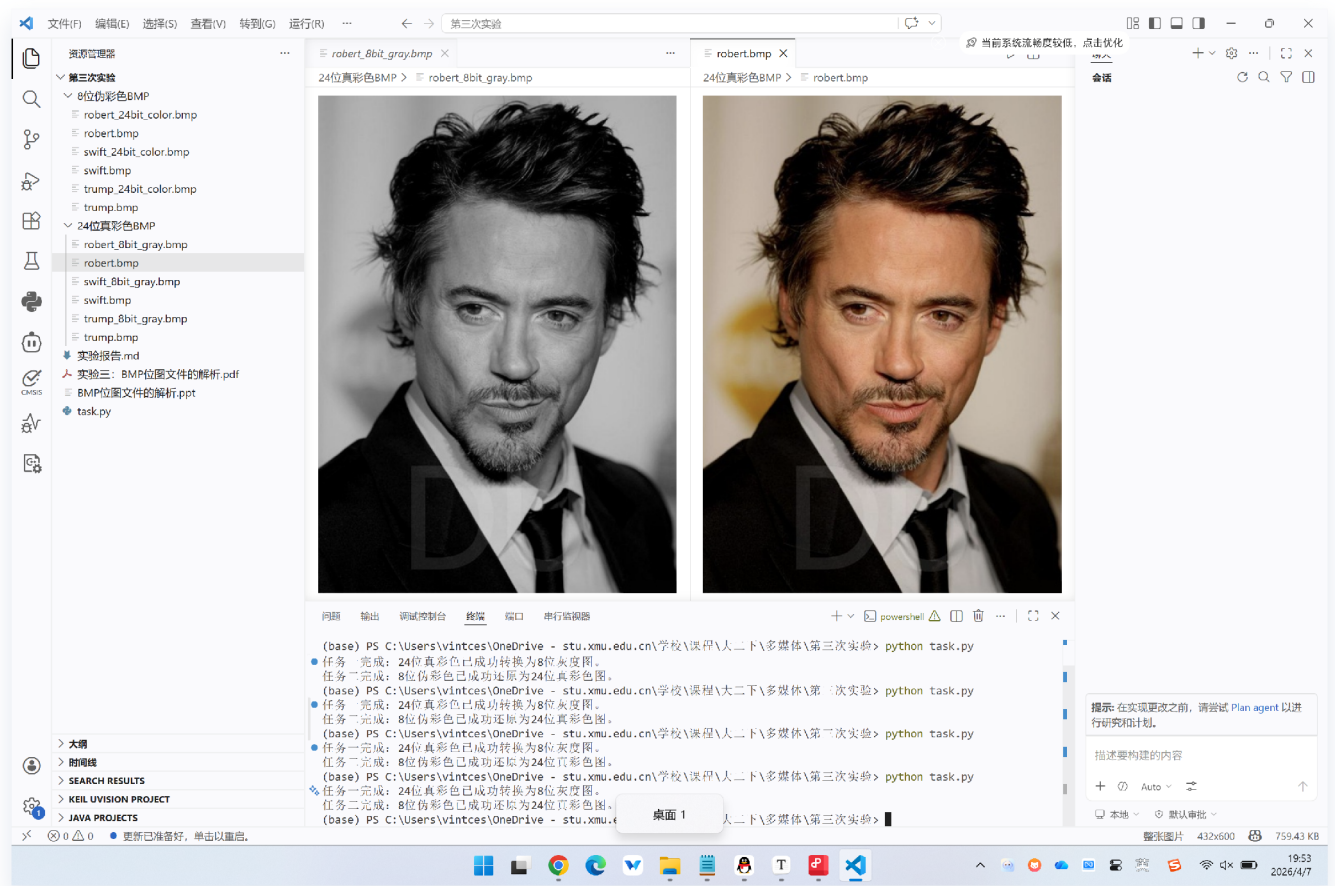
- 读取一个已提供的24位真彩色BMP文件素材。
- 遍历其二维像素矩阵，逐个提取出 B、G、R 通道数据。
- 应用心理学灰度公式（ $Y = 0.299R + 0.587G + 0.114B$ ）计算得到单一的灰阶值。
- 在内存中构建一个256级的灰度调色板（即 R=G=B=索引值）。
- 将计算得到的灰度值作为该灰度调色板的索引覆盖原像素区，并将类实例的位深修改为8，最后将其存储为8位伪彩色BMP文件。

- **8位伪彩色转24位真彩色任务：**

- 读取一个已提供的8位伪彩色BMP文件素材。
- 提取其1024字节的调色板数据并按组存入列表。
- 遍历像素矩阵中的每一个索引值，通过该索引反查调色板列表，还原出原本的 B、G、R 色彩数据。
- 清空调色板数据，将位深修改为24位，按像素重组数据流后最后存储为新的BMP文件。

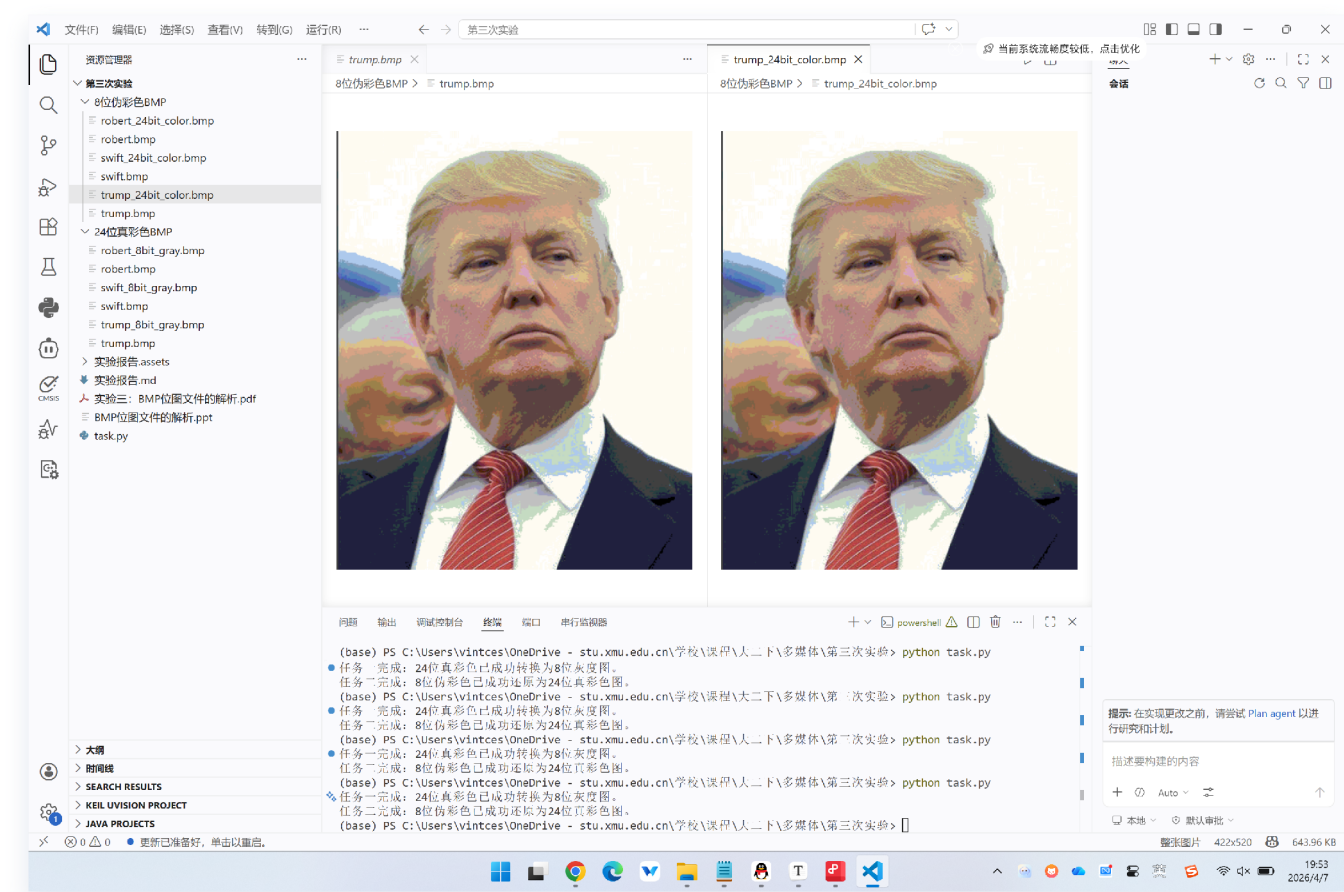
2. 运行程序截图和简要说明

2.1 24位真彩色转8位灰度效果



说明：程序成功读取24位真彩色BMP文件，将其像素数据按灰度公式进行逐点转换，并正确构建了256级调色板写入文件头后，生成了标准的单通道8位灰度图像，图像未发生错位扭曲。

2.2 8位伪彩色转24位真彩色效果



说明：程序成功解析了8位伪彩色BMP文件及其随附的调色板结构。通过索引查表的方式，准确提取出相应的 B、G、R 通道数据，并彻底移除了文件的调色板段，生成了色彩与原图一致的标准24位真彩色图像。

3. 核心代码展示和分析

3.1 24位转8位灰度核心逻辑

```
def convert_24_to_8_gray(self):  
    """任务一：读取24位真彩色，转化为灰色图像，存储为8位伪彩色"""  
    if self.bit_count != 24:  
        return  
  
    # 1. 创建包含256个灰阶的调色板（R=G=B=索引值）  
    self.palette = [(i, i, i) for i in range(256)]  
  
    # 2. 遍历像素利用心理学公式计算灰度，并将其作为索引  
    new_pixel_data = []  
    for row in self.pixel_data:
```

```

        new_row = []
        for b, g, r in row:
            gray = int(0.299 * r + 0.587 * g + 0.114 * b)
            new_row.append(gray) # 此处的灰度值恰好等于我们在调色板里创建的索引
        new_pixel_data.append(new_row)

self.pixel_data = new_pixel_data
self.bit_count = 8 # 修改位深标识

```

代码分析：本段代码针对24位原图无调色板的特性，在内存中动态生成了一个线性的256级灰阶调色板。在遍历像素阶段，通过加权公式将RGB三个通道压缩为一个标量，并直接将该值作为索引替换原始RGB元组，完成了从24位向8位的降维转换。

3.2 8位转24位真彩色核心逻辑

```

def convert_8_to_24(self):
    """任务二：读取8位伪彩色，转化为24位真彩色"""
    if self.bit_count != 8:
        return

    # 1. 遍历原有的索引像素矩阵，通过查调色板还原真实的 B, G, R 色彩值
    new_pixel_data = []
    for row in self.pixel_data:
        new_row = []
        for idx in row:
            new_row.append(self.palette[idx])
        new_pixel_data.append(new_row)

    # 2. 重新赋值像素区，清除不再需要的调色板，并修改位深标识
    self.pixel_data = new_pixel_data
    self.bit_count = 24
    self.palette = []

```

代码分析：该方法实现了调色板索引到原生色彩的反向映射。通过双层循环遍历由索引构成的二维列表，以索引为游标从 `self.palette` 中提取出对应的RGB元组。随后清空调色板状态并将位深标记为24，满足了真彩色文件的结构要求。

3.3 BMP底层字节解析与对齐保障

```
# --- 4. 解析像素数据区 ---
f.seek(bfOffBits)
# BMP要求图像每一行的字节数必须是4的倍数，计算对齐后的行字节数
row_padded = (self.width * self.bit_count + 31) // 32 * 4

self.pixel_data = []
for _ in range(abs(self.height)):
    row_data = f.read(row_padded)
    row_pixels = []
    if self.bit_count == 24:
        # 24位图直接读取 B, G, R
        for x in range(self.width):
            b, g, r = row_data[x*3], row_data[x*3+1], row_data[x*3+2]
            row_pixels.append((b, g, r))
    elif self.bit_count == 8:
        # 8位图读取的是调色板索引
        for x in range(self.width):
            row_pixels.append(row_data[x])
    self.pixel_data.append(row_pixels)
```

代码分析：这是脱离第三方库解析BMP图像时最核心的数据处理区。代码严格计算了 `(width * bit_count + 31) // 32 * 4` 这一边界公式，确保每次读取的一行数据能够跳过为了满足4字节对齐而填充的废弃字节，从而剥离出纯净的图像通道矩阵，避免了画面斜切问题。